Repaso Bases de Datos Relacionales (SQL)

Conceptos fundamentales

Las bases de datos relaciones se basan en un modelo de organización de los datos en:

- Tablas (o relaciones)
- Filas (o registros)
- Columnas (o campos)

Base de datos formada por tablas → Tablas compuestas por filas → Filas compuestas por una o más columnas.

Este modelo tiene 3 características:

- Estructura rígida → Filas de una tabla mismo esquema definido
- Integridad referencial → Relaciones entre las tablas por FK (Foreign Key)
- Normalización → Eliminar redundancia

Modelos de BD relacionales

Para definir esquemas o estructuras de tablas usamos DDL, esto nos permite definir y/o modificar la estructura de una tabla.

Siguiendo el esquema de una tabla podemos observar algunas particularidades:

- Establecemos una estructura fija → la tienen que seguir TODOS los registros de una tabla.
- Forma explícita de relacionar tablas. Se indica FK y CAMPO de la tabla que se relaciona.

Consultas en BD relacionales

Para leer datos desde una tabla usamos DQL, este lenguaje nos permite consultar datos ya registrados.

Tenemos 3 tipos de consultas:

- Consultas básicas → Se realizan sobre una tabla y no requieren de uso de otras que se encuentren relacionadas.
- Consultas multitablas → Se realizan en principio sobre una tabla pero se agregan datos pertenecientes a otras tablas relacionadas.
- Subconsultas → Son consultas alojadas dentro de otra consulta principal.

Manipulación de datos

Para manipular los datos (Creación, Modificación y Eliminación) utilizamos DML

- Creación → INSERT INTO
- Modificación → UPDATE
- Eliminación → DELETE

Introducción a BD No Relacionales (NoSQL)

¿Qué son?

Son todos los sistemas de BD que no siguen el modelo relacional tradicional

- SQL → Modelo Relacional
- NoSQL → Modelo No Relacional

Surgieron para solucionar limitaciones de las BD SQL principalmente por 3 motivos:

- Escalabilidad horizontal → Permite distribución de datos en nodos que conforman un clúster.
- Flexibilidad de los esquemas de datos → Convivencia de distintas estructuras de datos.
- Alto rendimiento → Consultas más rápidas.

Tipos de BD NoSQL

- BD de documentos: Se utiliza para almacenar y consultar datos semi-estructurados. Estos datos se almacenan en un documento similar al formato JSON o XML, el cual nos sirve como una representación directa del modelo de la aplicación.
  Ejemplos: → MongoDB, Apache CouchDB y Firebase Cloud Firestore.
- BD de pares clave - valor: Los datos se almacenan en un registro en el que una clave única (PK) se asocia a un valor, este puede ser una cadena de texto, un booleano o algún objeto.
  Ejemplos → Redis y Amazon DynamoDB
- BD de columnas: Los datos se almacenan como un conjunto de columnas. Cada una de ellas almacena un conjunto de valores donde cada uno corresponde a una fila lógica, la cual se infiere por la indexación del valor en la columna.
- BD de grafos: Los datos se organizan como nodos de un gráfico, enfocándose en las relaciones entre los datos. Esas conexiones entre los nodos se almacenan como entidades particulares (Elementos de primera clase) y no son tan solo resultado implícito de la conexión entre otras entidades.

Volviendo a MongoDB (BD documentales), vamos a ver cómo se organizan los datos:

MongoDB sigue una estructura jerárquica de componentes que se relacionan de la siguiente manera:

1. Clústers → agrupaciones de servidores o nodos.
2. Nodos → Servidores individuales que pueden, o no, tener distintas funcionalidades asociadas.
3. Bases de datos → Contenedores lógicos de datos dentro de un nodo.
4. Colecciones → Son análogas a las tablas del modelo relacional y agrupan los documentos de una entidad conceptual.
5. Documentos → Mínima unidad de almacenamiento en MongoDB. Equivalen a los registros del modelo relacional con un ID único y se almacenan en formato BSON (Binary JSON).

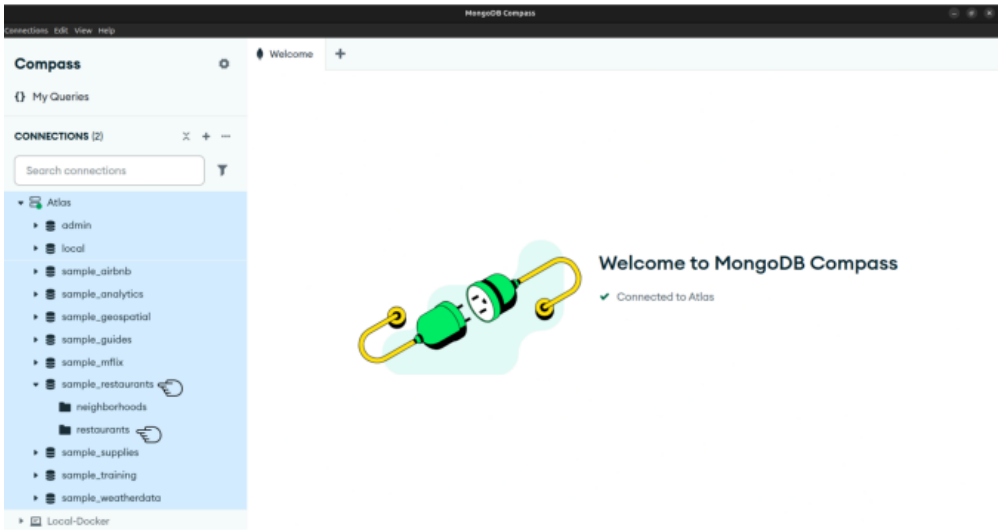
Una de sus principales características es la flexibilidad del esquema de los datos. Los campos dentro de los documentos de una colección son opcionales, este es el hecho que da lugar a la flexibilidad y, por ende, podemos hablar de polimorfismo de datos:

```
// Usuario básico
{
  "_id": ObjectId("..."),
  "nombre": "Ana García",
  "email": "ana@gmail.com",
  "tipo": "basico"
}

// Usuario premium
{
  "_id": ObjectId("..."),
  "nombre": "Carlos López",
  "email": "carlos@gmail.com",
  "tipo": "premium",
  "fecha_suscripcion": ISODate("2025-07-09T00:00:00Z"),
  "beneficios": ["envio_gratis", "soporte_24h"],
  "limite_credito": 5000
}

// Usuario corporativo
{
  "_id": ObjectId("..."),
  "razon_social": "Tech Solutions SA",
  "email": "contacto@techsolutions.com",
  "tipo": "corporativo",
  "empleados": 150,
  "contactos": [
    {
      "nombre": "María Rodriguez",
      "cargo": "Gerente de Compras",
      "telefono": "+54-11-1234-5678"
    }
  ]
}
```

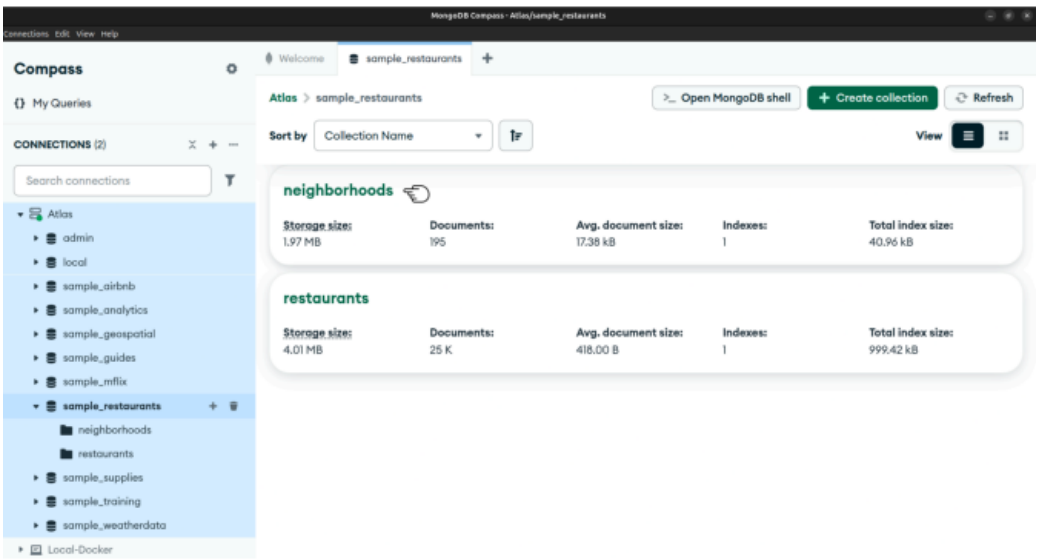
En el ejemplo todos los usuarios corresponden a la entidad → usuario, y por lo tanto pertenecen a la misma colección, pero al ser distintos tipos de usuarios la estructura varía por presencia o ausencia de datos opcionales. Por supuesto, organizarlos de esta manera tiene que tener sentido dentro del contexto del sistema en cuestión.

Gestores de BD NoSQL

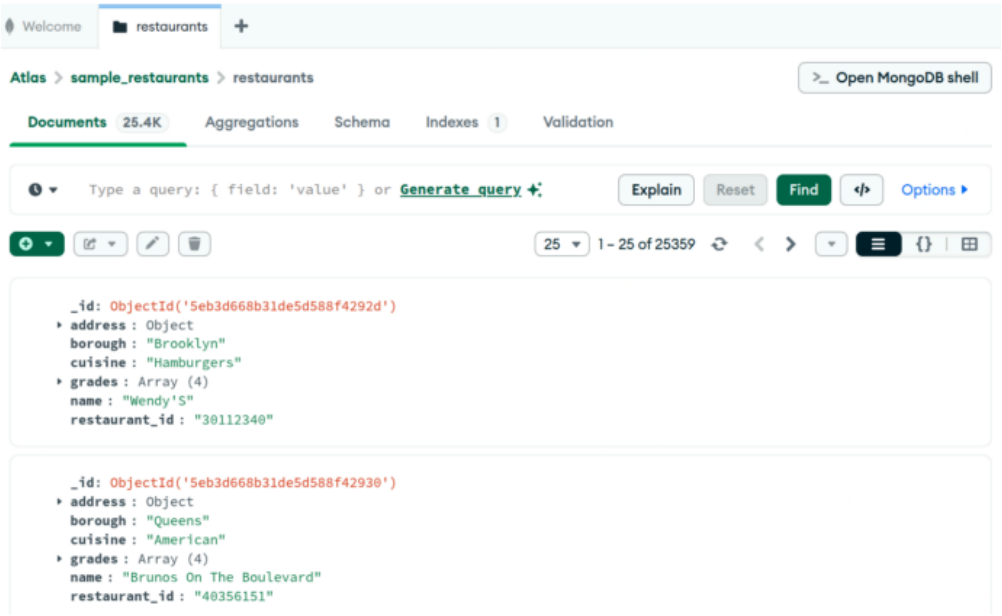
Para administrar las BD de MongoDB podemos utilizar el gestor MongoDB Compass. Este permite acceder, visualizar y manipular los datos almacenados en las bases de datos a través de una interfaz gráfica.



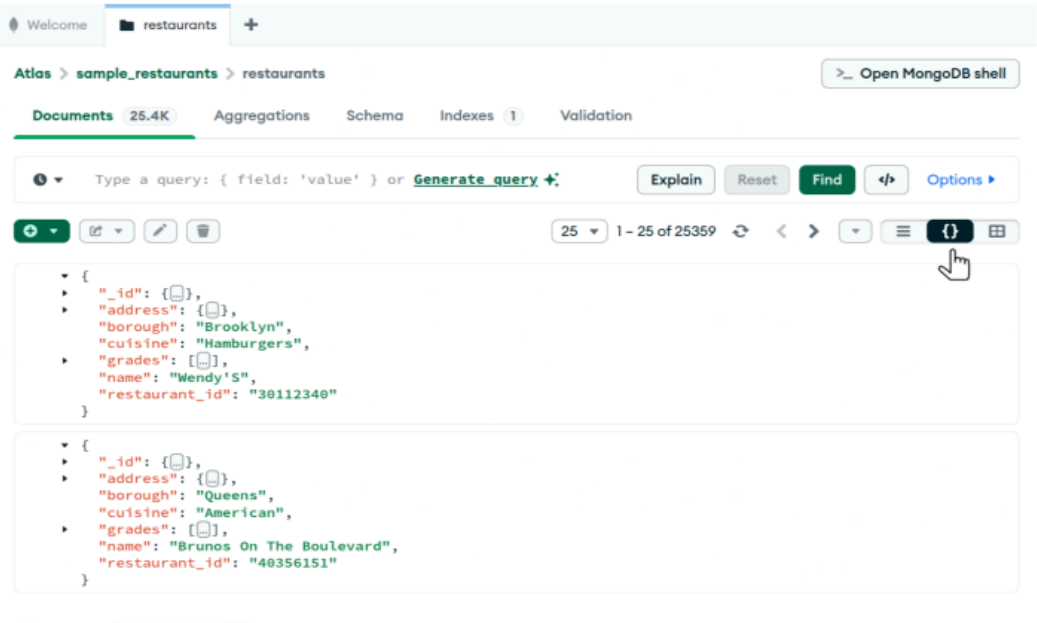
En esta captura se puede observar la interfaz que tiene MongoDB Compass, donde a la izquierda podemos ver dos clústers y el conjunto de bases de datos de uno de ellos. Las últimas pueden estar distribuidas entre nodos o replicarse entre los distintos nodos.
Podemos hacer “click” en alguna de las BD para ver sus colecciones con un resumen de los datos almacenados y su organización.

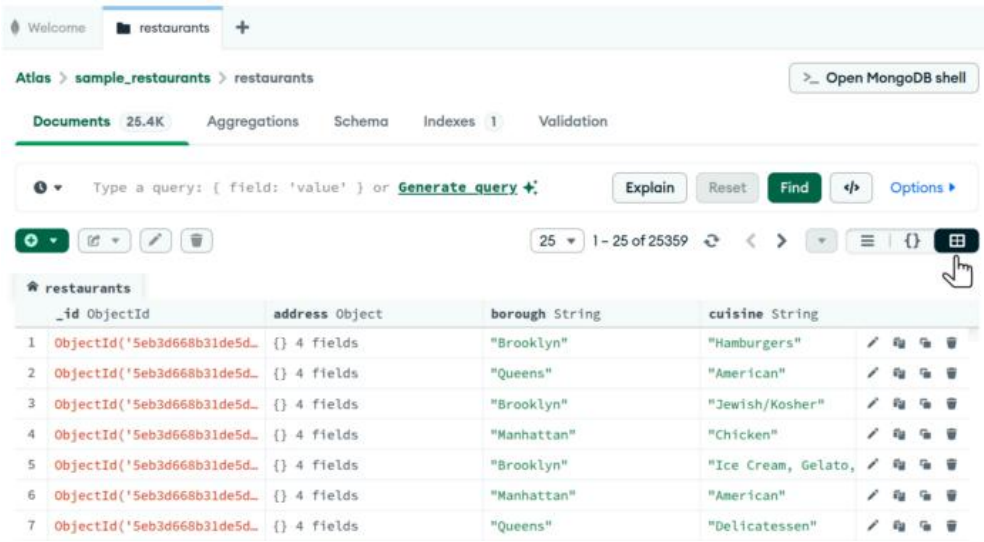


También se pueden observar los documentos de una de las colecciones haciendo “click” en una de ellas.

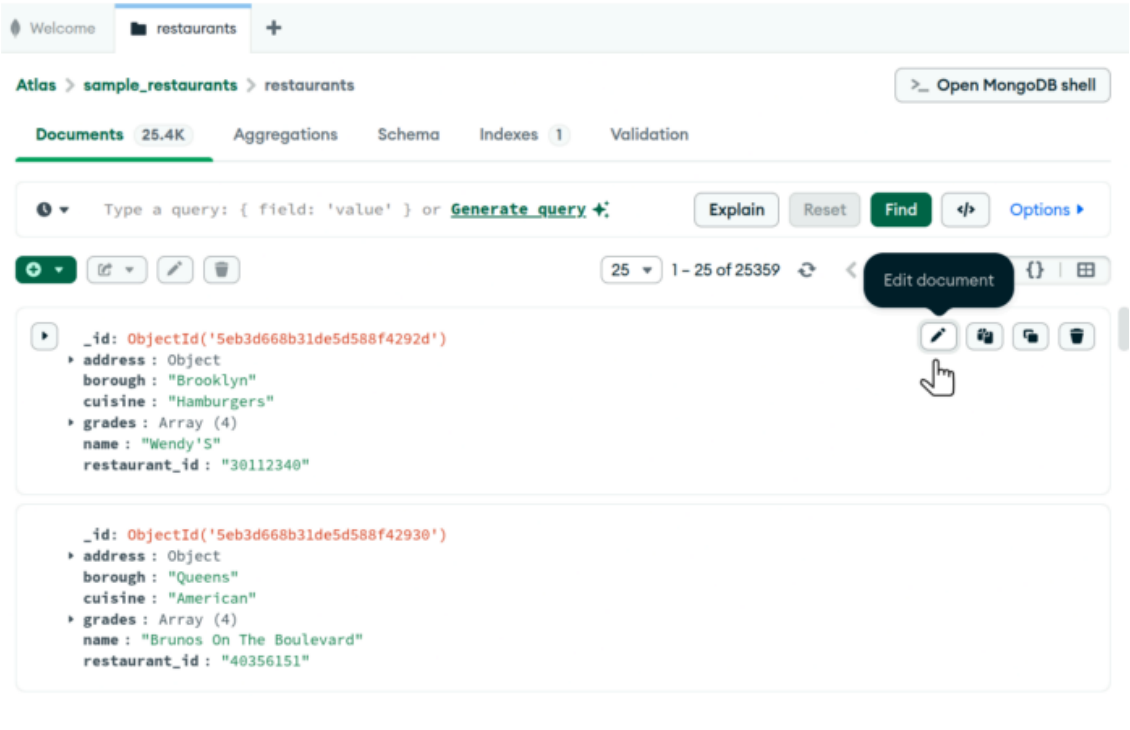


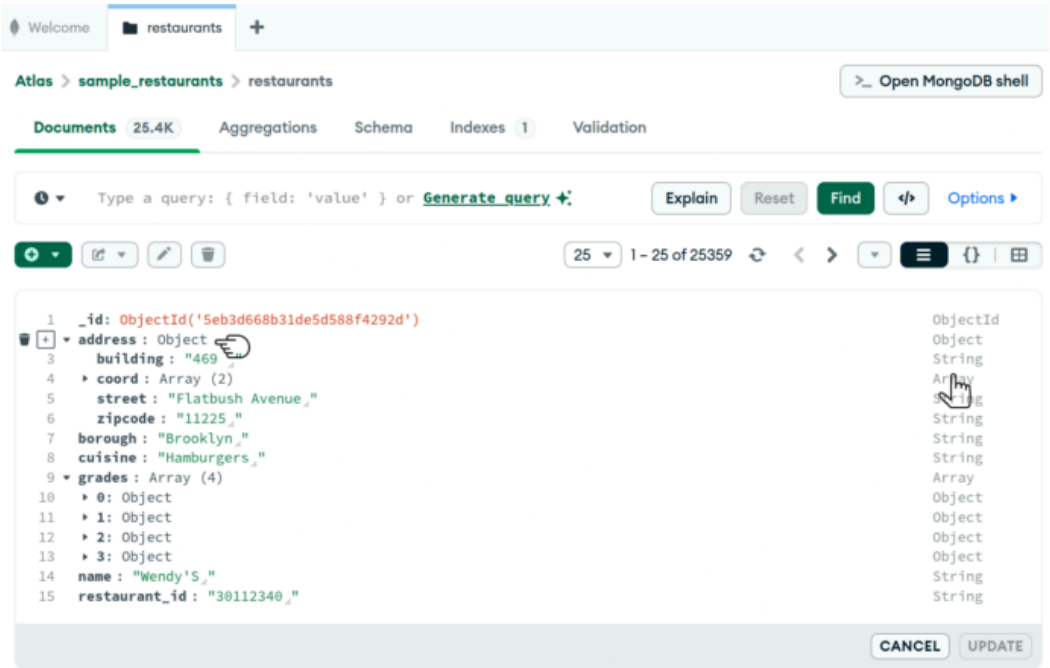
A esta altura llegamos a la mínima unidad de almacenamiento de MongoDB, que son los documentos, espacio que contiene los campos asociados para una entidad.





Es aquí donde podemos realizar consultas, manipular documentos, agregar nuevos o eliminarlos, y además jugar con las formas de visualización.
Si se quiere ir un poco más allá podemos hacer “click” en el ícono de edición que vemos en la esquina superior derecha de un documento, este nos permite editar un documento de manera directa a través de la GUI.

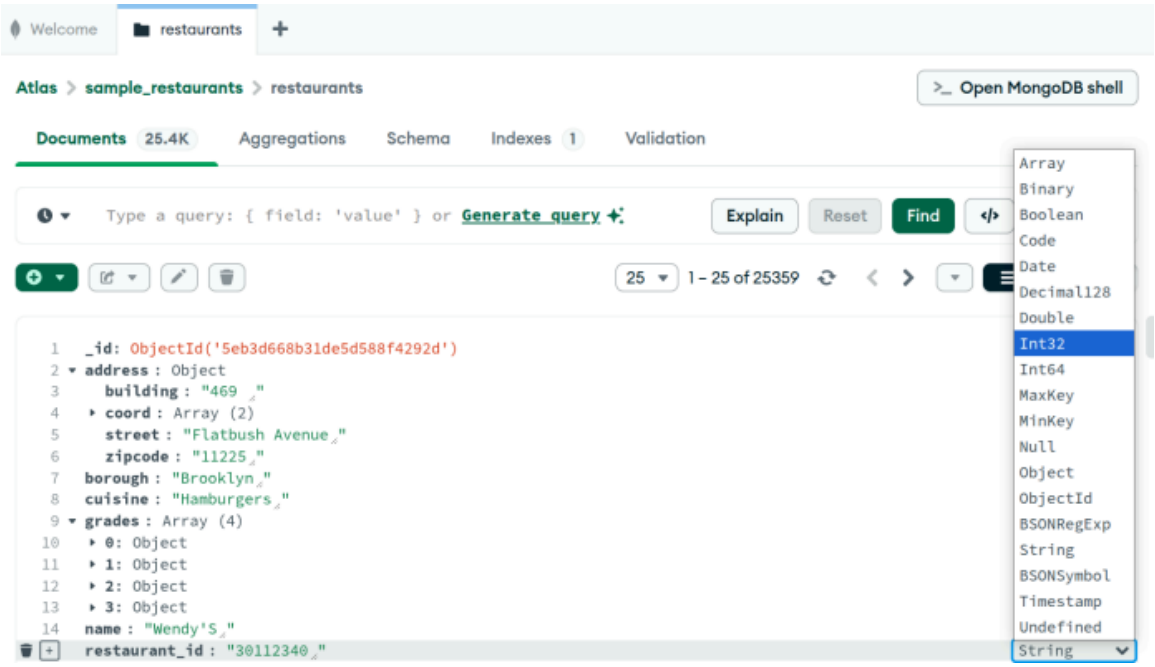




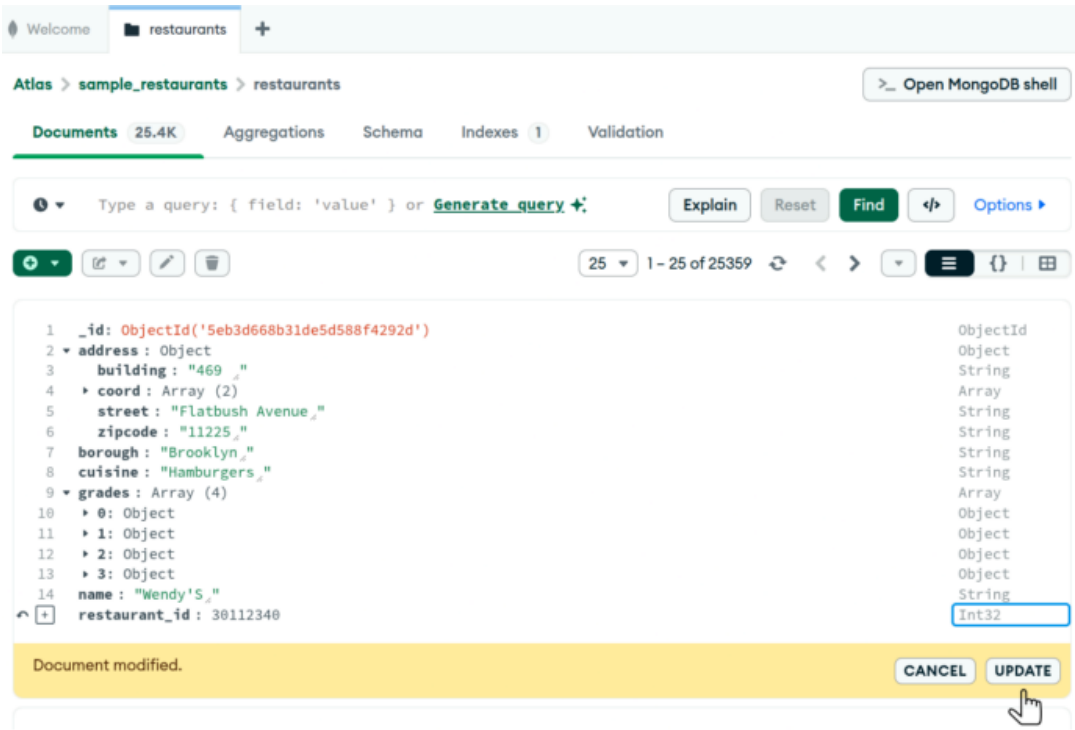
En esta captura podemos observar que, al habilitar la edición, se nos muestra el tipo de dato que almacena cada campo.

Tipos de dato soportado

Si investigamos qué pasa al intentar cambiar un tipo de dato para un campo parti cular, vemos que se despliega un listado con todos los tipos disponibles en MongoDB:



En primer lugar, se observa que son muchos tipos de datos más de los que admite el formato JSON debido a que el documento está en formato BSON, este último considerado como una extensión del formato anterior.
Ante cualquier cambio en un documento, es muy importante confirmar presionando el botón “UPDATE” (actualizar) que se habilitará luego de realizarse un cambio.



Comparaciones entre BD Relacionales (SQL) y No Relacionales (NoSQL)

Es primordial entender que las BD NoSQL no pretenden ser un reemplazo de las SQL. Cada una tiene contextos en los que es superior a la otra y para realizar la decisión sobre cuál usar se requiere de un análisis para evaluar cuál es la que mejor se adapte a una situación en particular.

Ventajas y desventajas de cada una:

Relacionales:

- Ventajas:
  → Esquema rígido, permite un formato unificado de datos, garantiza consistencia.
  → El estándar SQL es maduro y estable.
  → Garantizan una integridad referencial.
  → Son explícitas en cuanto a sus relaciones, brinda claridad.
  → Aseguran la atomicidad de las transacciones.
- Desventajas:
  → Escalabilidad limitada.
  → La rigidez del esquema dificulta los cambios.

No relacionales:

- Ventajas:
  → Esquema completamente flexible.
  → Escalabilidad horizontal.
  → Desempeño ágil y rápido.
- Desventajas:
  → Menor madurez y falta de estándar unificado.
  → La consistencia puede no ser la mejor.

Criterio de selección

Vamos a elegir una BD SQL cuando:

- Se requieren transacciones atómicas, que mantengan la consistencia entre los
- estados previo y posterior a la operación, que sean aisladas o independientes y que
- duren (ACID).
- Los datos deben mantener un esquema bien definido y debe ser estable.
- Se realicen consultas complejas de manera frecuente.
- La integridad de las relaciones sea crítica para el sistema en cuestión.

Por otro lado, vamos a elegir una BD NoSQL cuando:

- Necesitamos escalabilidad horizontal por crecimiento masivo.
- El esquema cambie en el tiempo.
- Los datos que se registren no sean estructurados.
- Haya un alto volumen de lecturas o escrituras.

Conclusiones

- Las bases relacionales siguen siendo la mejor opción para muchos casos de uso tradicionales.
- NoSQL ofrece ventajas significativas en escalabilidad y flexibilidad.
- La decisión debe basarse en requisitos específicos, no en modas tecnológicas.
- Es posible y común usar ambos tipos en una misma aplicación.